



# SunHarvest Agro

## Irrigação Solar Inteligente

Documentação Técnica — Componente IoT (ESP32)

---

Disruptive Architectures: IoT, IoB & Generative IA

FIAP — Análise e Desenvolvimento de Sistemas

**Global Solution 2026 - Grupo CCGL - Challenge 2025**

## 1. Visão Geral do Projeto

O SunHarvest Agro é um sistema de irrigação solar inteligente que combina dados de satélite da NASA com sensores IoT no campo para calcular com precisão quando e quanto irrigar. O problema central é o desperdício hídrico causado pela irrigação por achismo: sem dados locais de evapotranspiração, produtores irrigam em excesso (até 30% a mais que o necessário) ou causam estresse hídrico nas culturas.

A solução resolve dois problemas simultâneos:

- Cálculo da demanda hídrica via evapotranspiração FAO Penman-Monteith, usando dados da NASA POWER (radiação solar, temperatura, vento, umidade).
- Monitoramento da oferta energética da bomba solar, detectando falhas antes que a lavoura resseque.

## 2. Componente IoT — ESP32

O nó IoT é responsável pela coleta de dados físicos do campo que não podem ser obtidos por satélite: a umidade real do solo e a saúde do sistema de bombeamento solar. Ele se comunica com o backend Java via Wi-Fi usando uma API REST própria.

### 2.1 Hardware

| Componente      | Função                                  | Pino ESP32     | Tipo            |
|-----------------|-----------------------------------------|----------------|-----------------|
| Potenciômetro 1 | Simula umidade do solo (0–100%)         | GPIO 34        | ENTRADA         |
| Potenciômetro 2 | Simula corrente da bomba (0–10A)        | GPIO 35        | ENTRADA         |
| LED Verde       | Sistema OK (umidade e bomba normais)    | GPIO 26        | SAÍDA           |
| LED Vermelho    | Alerta (solo seco ou bomba com falha)   | GPIO 27        | SAÍDA           |
| LCD 16x2 I2C    | Interface local com dados em tempo real | SDA=21, SCL=22 | INTERFACE LOCAL |

### 2.2 Lógica de Funcionamento

A cada 3 segundos o ESP32:

- Lê o potenciômetro 1 e converte para umidade do solo (0–100%)
- Lê o potenciômetro 2 e converte para corrente da bomba (0–10A)
- Calcula a potência da bomba:  $P = \text{Tensão (24V)} \times \text{Corrente (A)}$
- Calcula a eficiência:  $(\text{Potência real} / \text{Potência nominal 1100W}) \times 100\%$
- Detecta falha se corrente  $< 2A$  ou  $> 8.5A$  ou eficiência  $< 40\%$
- Acende LED verde se tudo OK, vermelho se solo seco ou bomba com falha
- Atualiza o LCD alternando entre página de umidade e página de bomba
- Mantém histórico das últimas 10 leituras na memória

### 2.3 Simulação no Wokwi

O protótipo foi desenvolvido e validado no simulador Wokwi (wokwi.com), utilizando o modelo ESP32 DevKit V1 de 30 pinos. No hardware físico, o potenciômetro 1 seria substituído por um sensor de umidade capacitivo do solo, e o potenciômetro 2 pelo módulo INA219 para leitura real de tensão e corrente via I2C.

---

## 3. API REST — Endpoints

O ESP32 hospeda um WebServer na porta 80 com 4 endpoints. O backend Java (Spring Boot) consome GET /leitura periodicamente para obter dados do campo e POST /comando para acionar a bomba após calcular o ETo.

### 3.1 GET /

Retorna a identidade do nó IoT e configurações da fazenda cadastrada.

**Resposta de exemplo:**

```
{
  "device": "SunHarvest-IoT-Node",
  "farm_id": "farm-equipe07-gs2026",
  "crop_type": "Soja",
  "soil_type": "Franco",
  "area_ha": 5.0,
  "panel_w": 3000,
  "pump_w": 1100,
  "status": "online",
  "uptime_s": 3420,
  "bomba_ligada": false
}
```

### 3.2 GET /leitura

Retorna a leitura mais recente dos sensores. É o endpoint principal consumido pelo backend para calcular o ETo e decidir a irrigação.

**Resposta de exemplo:**

```
{
  "umidade_solo_pct": 24.3,
  "solo_seco": true,
  "corrente_bomba_a": 3.8,
  "tensao_painel_v": 24.0,
  "potencia_bomba_w": 91.2,
  "eficiencia_bomba_pct": 82.9,
  "bomba_falha": false,
  "bomba_ligada": true,
  "recomendacao_local": "IRRIGAR - solo abaixo do limite critico",
  "timestamp_ms": 45231
}
```

### 3.3 GET /historico

Retorna as últimas 10 leituras armazenadas em memória, permitindo ao backend análise de tendência ao longo do tempo.

#### Resposta de exemplo:

```
{
  "farm_id": "farm-equipe07-gs2026",
  "total_leituras": 7,
  "leituras": [
    {
      "umidade_solo_pct": 22.1,
      "corrente_bomba_a": 3.5,
      "potencia_bomba_w": 84.0,
      "eficiencia_bomba_pct": 76.4,
      "solo_seco": true,
      "bomba_falha": false,
      "timestamp_ms": 12000
    }
  ]
}
```

### 3.4 POST /comando

Recebe comandos do backend para acionar ou desligar a bomba de irrigação.

#### Corpo da requisição:

```
{ "acao": "ligar_bomba" }
```

ou

```
{ "acao": "desligar_bomba" }
```

#### Resposta:

```
{ "status": "bomba ligada" }
```

---

## 4. Integração com o Backend Java

O campo `iotDeviceId` no modelo `Farm` do backend Spring Boot armazena o IP do ESP32. O fluxo de integração funciona assim:

- O backend consulta GET /leitura a cada ciclo de monitoramento
- Combina umidade do solo com dados climáticos da NASA POWER API
- Calcula a evapotranspiração de referência (ET<sub>o</sub>) pelo método FAO Penman-Monteith
- Aplica o coeficiente de cultura (K<sub>c</sub>) específico para Soja em fase de crescimento
- Calcula a lâmina de irrigação necessária considerando a eficiência do sistema (85%)
- Se irrigação for necessária, envia POST /comando {"acao": "ligar\_bomba"} ao ESP32

- Registra tudo no banco PostgreSQL nas entidades IrradianceForecast, SoilReading e IrrigationAlert

## 4.1 NASA POWER API

URL base utilizada pelo backend para obter dados de irradiação e clima por localização:

```
GET https://power.larc.nasa.gov/api/temporal/daily/point
?parameters=ALLSKY_SFC_SW_DWN,T2M,WS2M,RH2M
&community=AG
&longitude=-46.63
&latitude=-23.55
&format=JSON
```

---

## 5. Como Executar a Simulação

### 5.1 Wokwi (ESP32)

- Acesse wokwi.com e crie um novo projeto ESP32
- Substitua sketch.ino pelo arquivo em /iot/sketch.ino
- Substitua diagram.json pelo arquivo em /iot/diagram.json
- Adicione as bibliotecas: LiquidCrystal I2C e ArduinoJson
- Clique em Play (▶) para iniciar a simulação
- Gire os potenciômetros para simular variações de umidade e corrente
- O IP do ESP32 aparece no LCD e no Serial Monitor

### 5.2 Dashboard

- Abra dashboard/index.html no navegador
- Digite o IP do ESP32 e clique em CONECTAR
- Ou clique em DEMO para ver o dashboard com dados simulados

### 5.3 Bibliotecas Necessárias (libraries.txt)

```
LiquidCrystal I2C
ArduinoJson
```